

Basic JavaScript

1. What are the different data types in JavaScript?

JavaScript has 8 data types:

 **Primitive Data Types** (Immutable & stored by value):

1. **String** - Represents text
2. **Number** - Any numeric value (integer or float)
3. **Boolean** - `true` or `false`
4. **Undefined** - A variable declared but not assigned a value.
5. **Null** - Represents intentional absence of value
6. **Symbol** - A unique, immutable value (used for object keys)
7. **BigInt** - For large integers beyond Number limit

 **Non-Primitive Data Type** (Mutable & stored by reference):

8. **Object** - Used to store collections of data (Includes arrays, functions, dates, etc.)

Tip: Use `typeof` to check a data type

2. What is the difference between `var`, `let`, and `const`?

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function-scoped	Block-scoped	Block-scoped
Reassignable	✓ Yes	✓ Yes	✗ No
Redeclarable	✓ Yes	✗ No	✗ No
Hoisted	✓ (but undefined)	✓ (not initialized)	✓ (not initialized)

3. What is hoisting in JavaScript?

👉 Hoisting is JavaScript's default behavior of **moving declarations to the top** of the current scope during compilation.

Example:

```
console.log(x); // undefined
var x = 5;
```

Only declarations are hoisted, not initializations.

4. Explain the difference between `==` and `===`.

- `==` (loose equality): compares values **after type conversion**.
- `===` (strict equality): compares values **and types**, no conversion.

Example:

```
'5' == 5    // true
'5' === 5   // false
```

5. What are truthy and falsy values?

- **Truthy:** Values that evaluate to `true` in a boolean context.
- **Falsy:** Values that evaluate to `false`. Only **7 falsy values**:

```
(false, 0, "", null, undefined, NaN,
document.all)
```

Everything else is **truthy** (e.g., `"0"`, `[]`, `{}`).

6. What is the difference between `null` and `undefined`?

Feature	<code>undefined</code>	<code>null</code>
Meaning	Variable declared but not assigned	Explicitly set to "no value"
Type	<code>undefined</code>	<code>object</code> (legacy bug)
Set by	JavaScript engine	Developer

7. What is a closure in JavaScript?

👉 A **closure** is a function that **remembers variables** from its outer scope even after the outer function has finished executing.

Example:

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    return count;  
  };  
}  
  
const counter = outer();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

8. Explain scope (global vs local vs block).

- **Global Scope:** Variables declared outside any function.
- **Function (Local) Scope:** Variables declared inside a function.
- **Block Scope:** Variables declared using `let` or `const` inside `{}`.

Example:

```
if (true) {  
    let x = 10; // block scope  
    var y = 20; // function/global scope  
}  
console.log(y); // 20  
// console.log(x); // Error
```

9. What is the event loop in JavaScript?

👉 The **event loop** handles asynchronous operations like callbacks, promises, and events.

Process:

1. Executes synchronous code.
2. Moves async tasks to the **callback queue** or **microtask queue**.
3. Pushes them to the call stack when it's empty.

This allows JavaScript to be **non-blocking** despite being single-threaded.

10. What is a callback function?

👉 A **callback** is a function passed as an argument to another function, to be **called later**, usually after an asynchronous operation.

Example:

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}
function sayBye() {
  console.log("Goodbye!");
}
greet("John", sayBye);
```

🎯 Functions and Objects

1. What is the difference between function declaration and function expression?

- **Function Declaration**

Defined with the `function` keyword and **hoisted**.

```
function greet() {
  console.log("Hello");
}
```

- **Function Expression**

Stored in a variable. **Not hoisted**.

```
const greet = function() {
  console.log("Hello");
};
greet();
```

2. What is a higher-order function?

👉 A function that:

- Takes another function as an argument, **or**
- Returns another function.

Examples: `map()`, `filter()`, `reduce()`, etc.

```
function double(x) {  
  return x * 2;  
}  
function apply(fn, value) {  
  return fn(value); // higher-order  
}  
apply(double, 5); // 10
```

3. What is the difference between `map()`, `forEach()`, `filter()`, and `reduce()`?

Method	Purpose	Returns	Mutates Original?
<code>map()</code>	Transforms each item	New array	✗ No
<code>forEach()</code>	Loops through items	<code>undefined</code>	✗ No
<code>filter()</code>	Keeps items that pass a test	New filtered array	✗ No
<code>reduce()</code>	Reduces array to a single value	Single result	✗ No

Example:

```
const nums = [1, 2, 3, 4];  
nums.map(x => x * 2); // [2, 4, 6, 8]
```

```

nums.filter(x => x > 2);    // [3, 4]
nums.reduce((a, b) => a + b); // 10
nums.forEach(x => console.log(x)); // 1 2 3
4

```

4. What is the difference between `call()`, `apply()`, and `bind()`?

Method	Purpose	Parameters	Executes Function?
<code>call()</code>	Calls a function with a given <code>this</code>	Arguments passed individually	✓ Yes
<code>apply()</code>	Same as <code>call()</code> but takes arguments as an array	Arguments passed as array	✓ Yes
<code>bind()</code>	Returns a new function with bound <code>this</code>	Arguments passed individually	✗ No

Example:

```

function sayHello() {
  console.log(this.name);
}

const user = { name: "John" };
sayHello.call(user); // John
sayHello.apply(user); // John
const boundHello = sayHello.bind(user);
boundHello(); // John

```

5. What is **this** keyword in JavaScript?

👉 **this** refers to the **object that is calling** the function.

Behavior in different contexts:

- **Global scope (non-strict):** **this** is the **window** object.
- **Inside a method:** **this** refers to the object.
- **In arrow functions:** **this** is taken from the surrounding (lexical) scope.
- **Inside a constructor:** **this** is the newly created object.

Example:

```
const obj = {
  name: "John",
  greet() {
    console.log(this.name); // John
  }
};
obj.greet();
```

6. Explain prototypal inheritance.

👉 JavaScript uses **prototypes** to allow objects to inherit features from other objects.

- Every object has an internal **[[Prototype]]** (accessed using **__proto__** or **Object.getPrototypeOf()**).
- Objects can share methods via the prototype chain.

Example:

```
const parent = { sayHello() {
  console.log("Hello!");
}
```

```
};
const child = Object.create(parent);
child.sayHello(); // Inherited from parent
```

7. What is JSON and how is it used in JavaScript?

👉 JSON (JavaScript Object Notation) is a **lightweight data format** used to store and transfer data.

- Used in APIs, localStorage, and config files.

Converting:

```
const obj = { name: "John" };
const jsonStr = JSON.stringify(obj); // to
JSON string

const backToObj = JSON.parse(jsonStr); //
back to JS object
```

8. What is the difference between a **shallow** and **deep copy**?

Copy Type	Description	Example Tools
Shallow Copy	Copies only the first level of data	<code>Object.assign()</code> , <code>spread ...</code>
Deep Copy	Copies nested objects/arrays completely	<code>JSON.parse(JSON.stringify(obj))</code> , <code>structuredClone()</code>

```
// Shallow copy
const original = { a: 1, b: { c: 2 } };
const shallow = { ...original };
shallow.b.c = 99; // Also changes original.b.c
```

```
// Deep copy
const deep = JSON.parse(JSON.stringify(original));
deep.b.c = 200; // Does not affect original
```

9. What is the difference between `Object.freeze()` and `Object.seal()`?

`Object.freeze()`

- Makes the entire object **immutable**.
-  You **cannot add, remove, or modify** any properties.
- Properties become **non-writable** and **non-configurable**.

`Object.seal()`

-  You **can modify existing properties**.
-  You **cannot add or delete** properties.
- Properties become **non-configurable**, but still **writable**.

Asynchronous JavaScript

1. What are **Promises**? How do they work?

 A **Promise** is an object that represents the **eventual result** (or failure) of an asynchronous operation.

Example:

```
const promise = new Promise((resolve,
reject) => {
  setTimeout(() => {
    resolve("Done!");
  }, 1000);
});
```

```
promise.then((result) =>
  console.log(result)); // "Done!"
```

- You create a promise with `new Promise((resolve, reject) => {})`
- You handle the result using `.then()` for success, `.catch()` for error.

2. What is **async/await**? How is it different from **Promises**?

👉 **async/await** is **syntax sugar** over Promises that makes asynchronous code look and behave more like synchronous code.

Example:

```
async function fetchData() {
  const result = await
  fetch("https://api.example.com");
  const data = await result.json();
  console.log(data);
}
```

Difference from Promises:

Feature	Promise	Async/Await
Style	<code>.then()/.catch()</code>	<code>try/catch</code> , more readable
Error handling	<code>.catch()</code>	<code>try/catch</code> block
Chaining	Chain methods	Use <code>await</code> in sequence

3. What are the different states of a **Promise**?

👉 A Promise has **3 states**:

1. **Pending** - initial state, not resolved or rejected.
2. **Fulfilled** - operation completed successfully (**resolve** was called).
3. **Rejected** - operation failed (**reject** was called).

Example:

```
const promise = new Promise((resolve, reject) => {  
  // pending  
  resolve("Success"); // fulfilled  
  // reject("Error"); // rejected  
});
```

4. What is the difference between **synchronous** and **asynchronous** code?

Synchronous Code	Asynchronous Code
Runs line by line	Doesn't block next lines
Slower if tasks take time	Non-blocking, faster execution
Example: Simple math	Example: <code>setTimeout()</code> , <code>fetch</code>

Example:

```
// Synchronous  
console.log("1");  
console.log("2");  
  
// Asynchronous  
console.log("1");  
setTimeout(() => console.log("2"), 1000);  
console.log("3"); // Output: 1, 3, 2
```

5. What is **callback hell** and how can it be avoided?

👉 **Callback Hell** happens when many asynchronous functions are nested inside each other, making code hard to read and maintain.

Example:

```
//callback hell
doTask1(() => {
  doTask2(() => {
    doTask3(() => {
      console.log("All done!");
    });
  });
});
```

How to avoid it:

-  Use Promises
-  Use async/await
-  Use named functions instead of inline callbacks

ES6+ Concepts

1. What are template literals?

👉 Template literals are strings wrapped in **backticks** (```) that allow **embedded expressions** using `${}`.

Example:

```
const name = "John";
const greeting = `Hello, ${name}!`;
console.log(greeting); // Hello, John!
```

They also support multi-line strings without needing `\n`.

2. What are arrow functions and how are they different from regular functions?

👉 Arrow functions are a **shorter syntax** for writing functions, introduced in ES6.

Example:

```
const add = (a, b) => a + b;
```

Key Differences:

Feature	Arrow Function	Regular Function
<code>this</code> behavior	Lexically bound (no own <code>this</code>)	Dynamically scoped
Syntax	Concise	Verbose
Usage as constructor	❌ Not allowed	✅ Allowed

3. What are rest and spread operators?

Both use `...` but for different purposes:

Rest operator: gathers items into an array

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b);  
}
```

Spread operator: expands arrays or objects

```
const arr = [1, 2, 3];  
const copy = [...arr]; //[1, 2, 3]  
  
const obj1 = { a:1 };
```

```
const obj2 = { ...obj1, b:2 }; //{ a:1, b:2
}
```

4. What is destructuring in JavaScript?

👉 Destructuring allows you to **unpack values** from arrays or objects into variables.

Examples:

```
const arr = [1, 2];
const [a, b] = arr; // a = 1, b = 2
const obj = { name: "Ganesh", age: 25 };
const { name, age } = obj;
```

5. What are default parameters?

👉 Allows function parameters to have **default values** if not provided.

Example:

```
function greet(name = "Guest") {
  console.log(`Hello, ${name}`);
}
greet(); // Hello, Guest
greet("John"); // Hello, John
```

6. What are modules in JavaScript (import/export)?

👉 **Modules** let you **break your code** into reusable pieces using **export** and **import**.

In **math.js**:

```
export const add = (a, b) => a + b;
```

In **main.js**:

```
import { add } from './math.js';
console.log(add(2, 3)); // 5
```

- **export** – makes variables/functions available outside the file.

- `import` — brings them into another file.
 Used in both frontend (ES6) and backend (Node.js with ESM or `require` in CommonJS).